Retrieval-augmented generation

CS 485, Spring 2026
Applications of Natural Language Processing

Katrin Erk
College of Information and Computer Sciences
University of Massachusetts Amherst

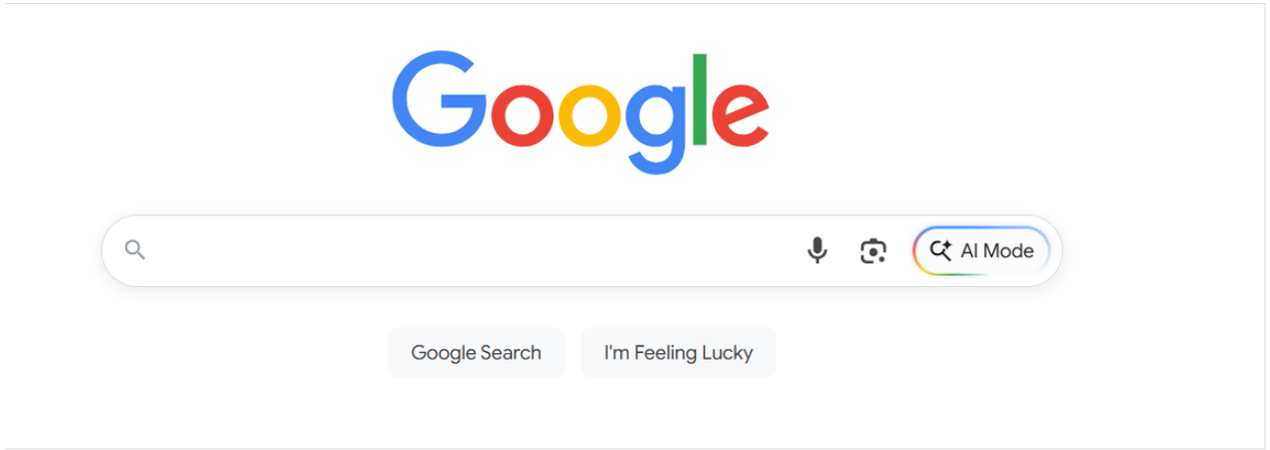
With slides from Jurafsky& Martin

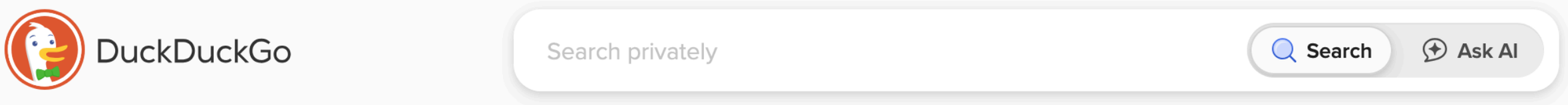
Retrieval

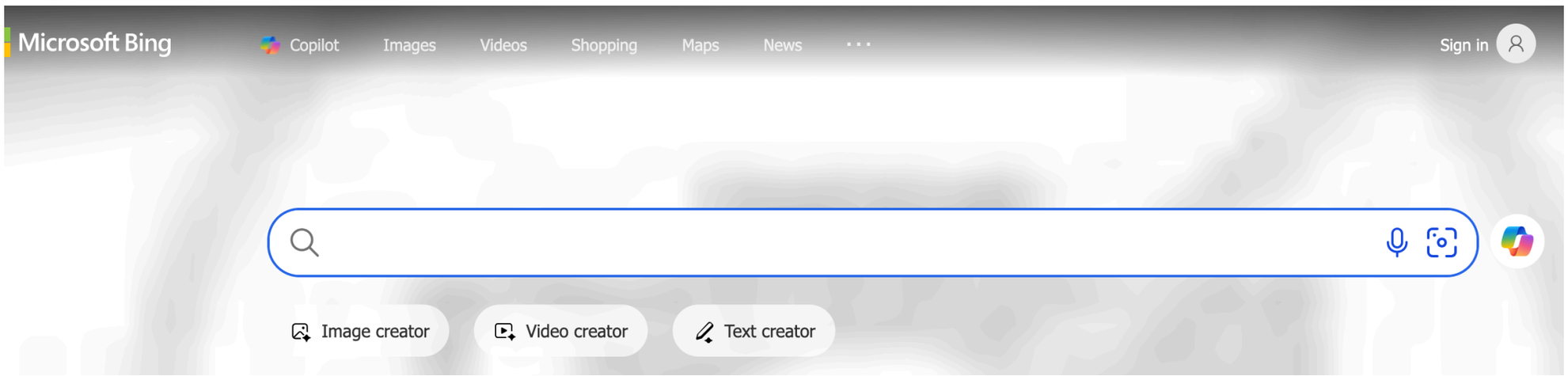
Information retrieval

- User has an information need
- And has some collection of documents
- User wants to find a relevant document
  - a document (or documents)
  - in the collection
  - that satisfy their need

Web search


Google
AI Mode
Google Search
I'm Feeling Lucky


DuckDuckGo
Search privately
Search
Ask AI


Microsoft Bing
Copilot
Images
Videos
Shopping
Maps
News
Sign in
Image creator
Video creator
Text creator

Not just the web

- Searching our email
- Searching corporate documents
- Searching personal medical records
- And also, part of LLMs
  - retrieval-augmented generation (RAG)

A bird’s eye view of RAG

IR plays a central role in modern LLMs

- How to answer factual questions like:
  - Where is the Louvre Museum located?
  - How to get a script D in LaTeX?
  - Where does the energy in a nuclear explosion come from?

Just prompt an LLM

KE **Katrin Erk**

Where is the Louvre Museum located?

2579 tokens

OpenAI GPT-5-mini 🔍 🖼️ 📄

The Louvre Museum is in Paris, France — in the 1st arrondissement on the Right Bank of the Seine. Its main entrance is at the Palais du Louvre (near the glass Pyramid on the Cour Napoléon, Rue de Rivoli, 75001 Paris).

128 tokens

LLMs seem to store facts in the weights
in their feedforward layers

But there are problems

LLMs hallucinate

Hallucination: a response that is not faithful to the facts of the world.

In the legal domain LLMs were shown to hallucinate between 58% of the time with ChatGPT-4 and 88% of the time with Llama 2

Dahl et al 2024, paper link
https://arxiv.org/abs/2401.01301

Can't use Proprietary Data

People need to ask questions about:

- personal email.
- healthcare applications to medical records.
- internal corporate documents
- legal documents discovery

Can't Handle Dynamic Data

LLMs can't answer questions about rapidly changing information

For example: Things that happened last week

In general, data shifts over time

Solution: RAG

Retrieval-Augmented Generation

1.Use IR to retrieve documents from some collection

2.Then use LLM to generate an answer conditioned on the documents

More about retrieval

Two architectures

Sparse retrieval

- represent query and doc as vectors of word counts
- weighted, e.g. by tf-idf

Dense retrieval

- Use LLM to represent query and document as embeddings

In both cases, similarity is dot product or cosine between query and document representations

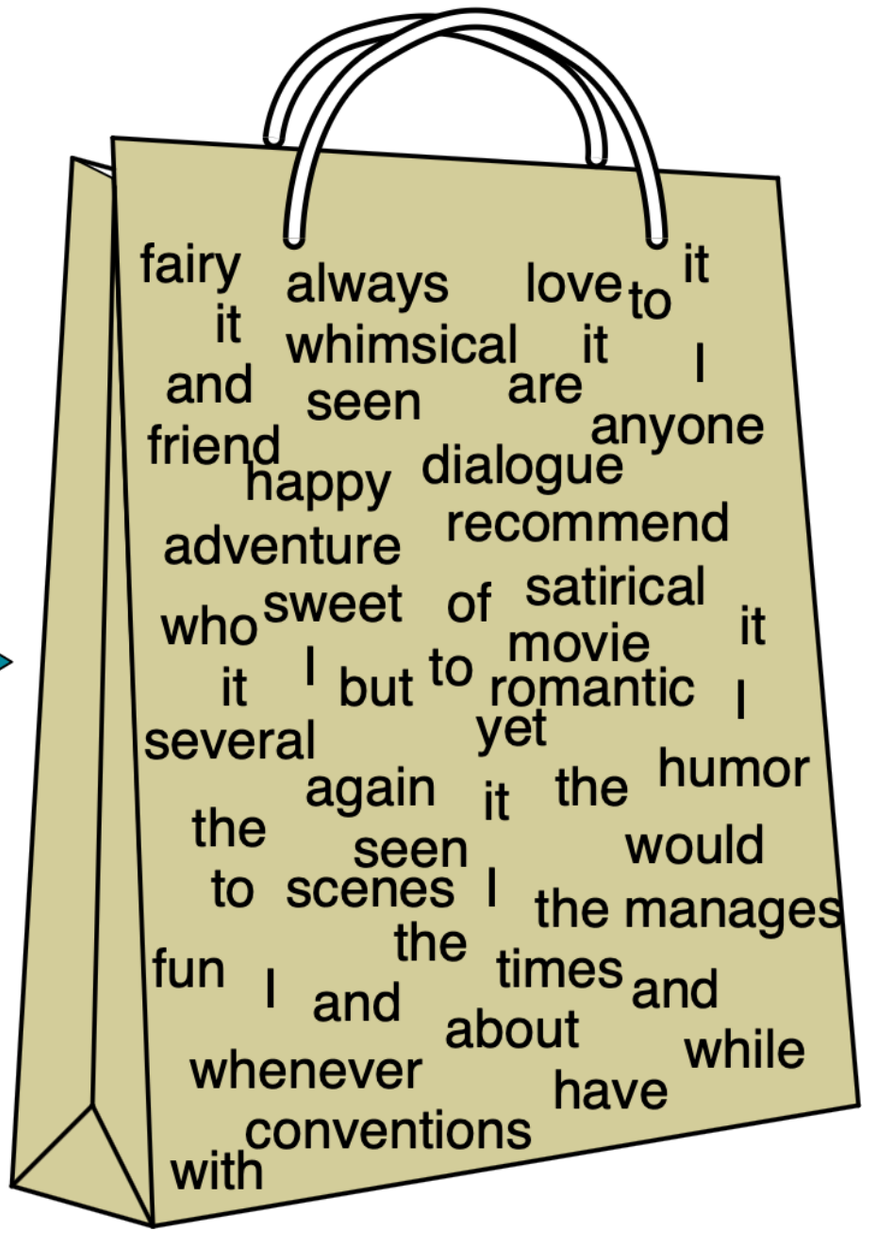
Sparse retrieval

The vector space model of IR:
Represent a document as a vector of counts of the words it contains.

Gerard Salton, 1971

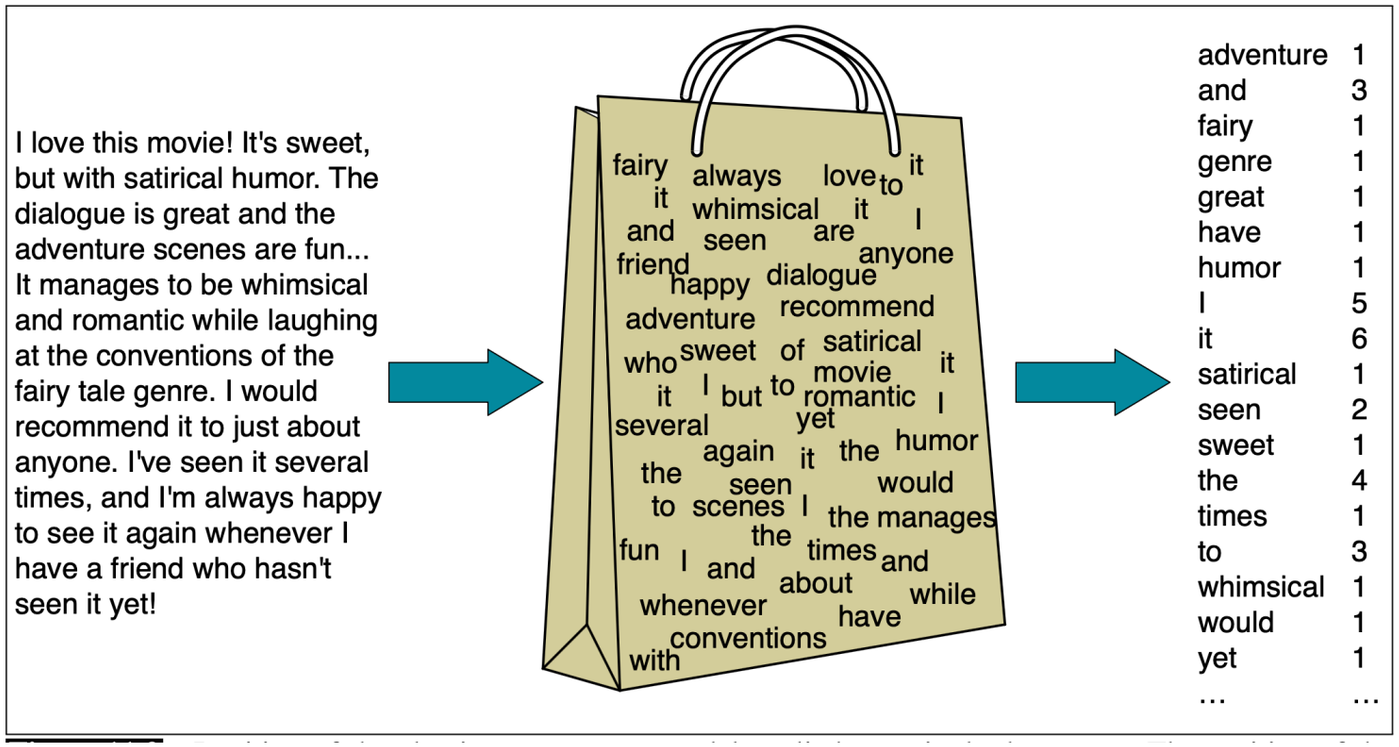
A document as a bag of words

I love this movie! It's sweet, but with satirical humor. The dialogue is great and the adventure scenes are fun... It manages to be whimsical and romantic while laughing at the conventions of the fairy tale genre. I would recommend it to just about anyone. I've seen it several times, and I'm always happy to see it again whenever I have a friend who hasn't seen it yet!



adventure	1
and	3
fairy	1
genre	1
great	1
have	1
humor	1
I	5
it	6
satirical	1
seen	2
sweet	1
the	4
times	1
to	3
whimsical	1
would	1
yet	1
…	…

A document as a bag of words



Vector representation of that document:

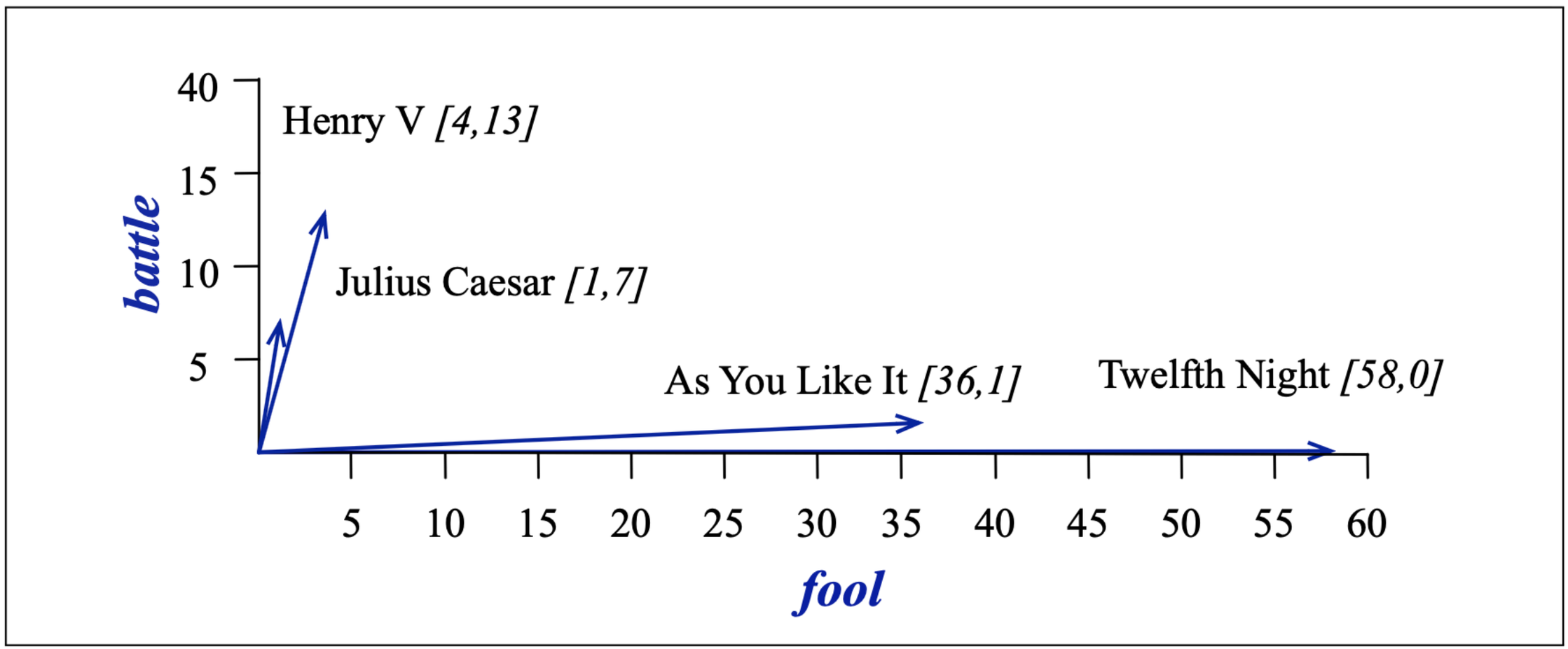
[1 3 1 1 1 1 1 5 6 1 2 1 4 1 3 1 1 1]

Term-document matrix

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

A matrix with a column for each document,
and a row for each content word.
Each document is represented by a vector of words.
Example here: 4 Shakespeare plays

Documents as points in space



Here: Four Shakespeare plays as points in the two-dimensional space with dimensions “battle”, “fool”

Vector similarity among documents

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

As You Like It, Twelfth Night are comedies,
the other two are not.

The two comedies are similar:
They have more fools and wit, and fewer battles.

Vector similarity among documents

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

Say we are looking for a witty fool play.

Query: “fool wit”, vector <0, 0, 1, 1>

Then we can retrieve documents by cosine similarity of document vector and query vector

Efficient retrieval: “inverted index”

- Mapping from search term to documents that contain it
- Can also contain additional information:
  - Frequency of the search term in the document
  - Position of the search term in the document

Dense retrieval

A problem with classic IR

- The vocabulary mismatch problem
- Sparse retrieval cosine similarities only work if there is exact word overlap between query and document
- But the query-writer cannot know the exact words the document might include

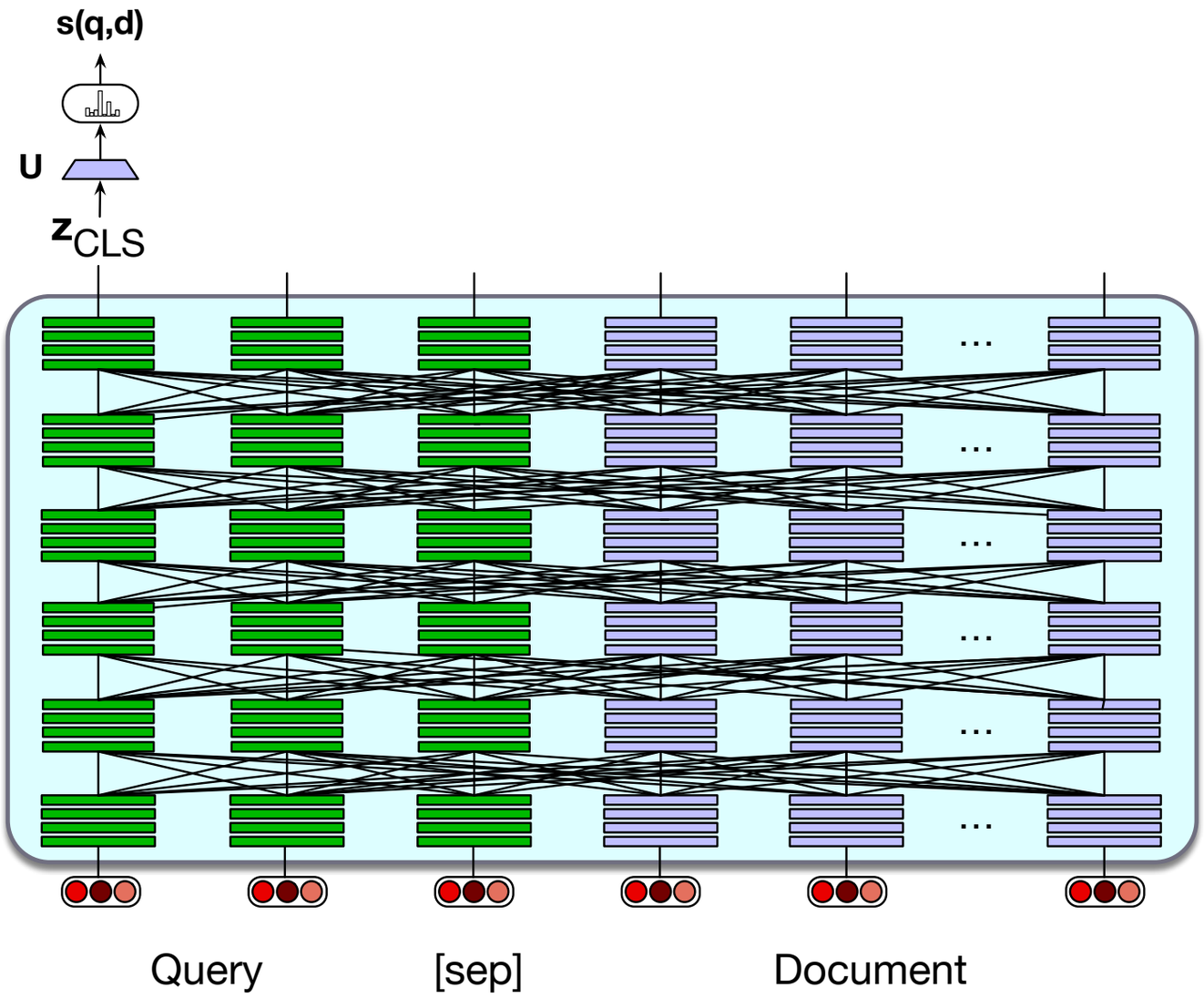
Dense retrieval

- Instead of representing query and documents with count vectors
- Represent both with embeddings

Hypothetical version of dense retrieval: static embeddings

- Replace count-based vectors with, e.g., word2vec
  - Query: the mean of the embeddings of each query word
  - Document: the mean of the document word embeddings
  - Then compute query-document cosine as normal.
- We don't do this because **contextual embeddings** work much better

Dense retrieval #1: Single encoder

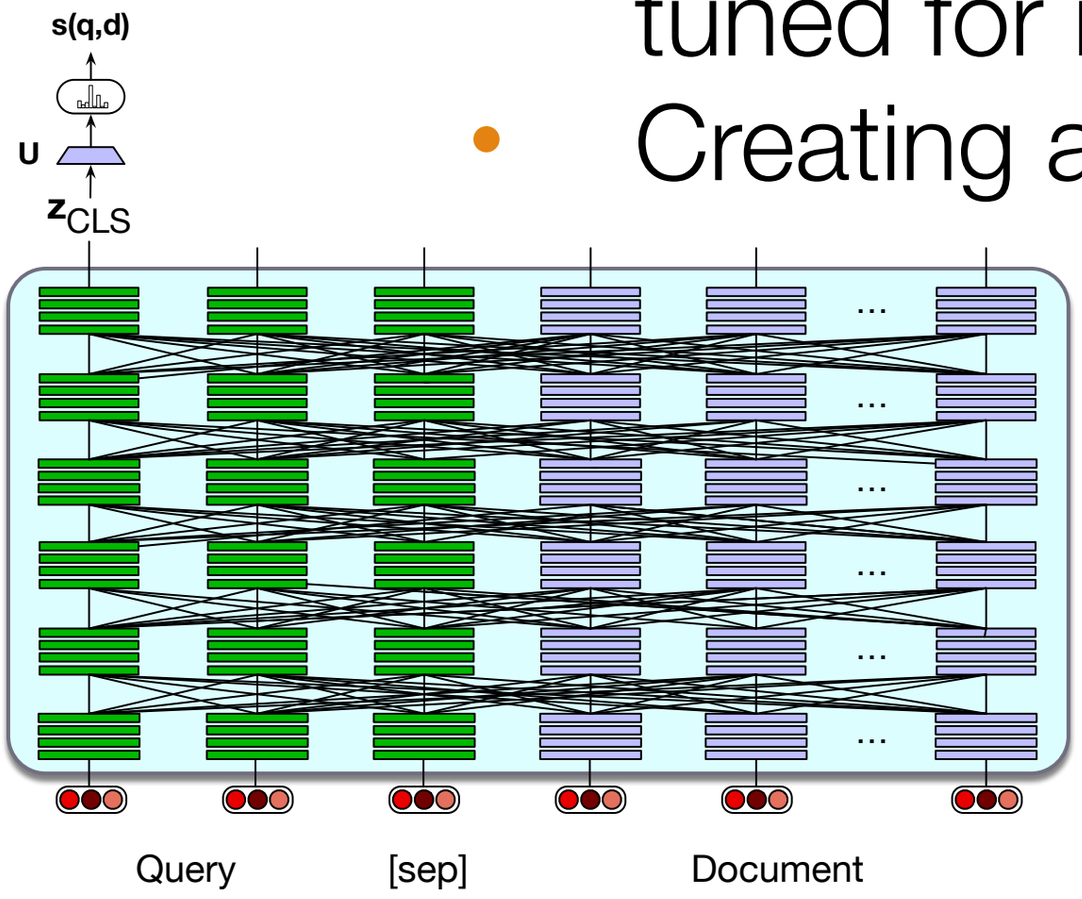


z = BERT(q; [SEP];d) [CLS]

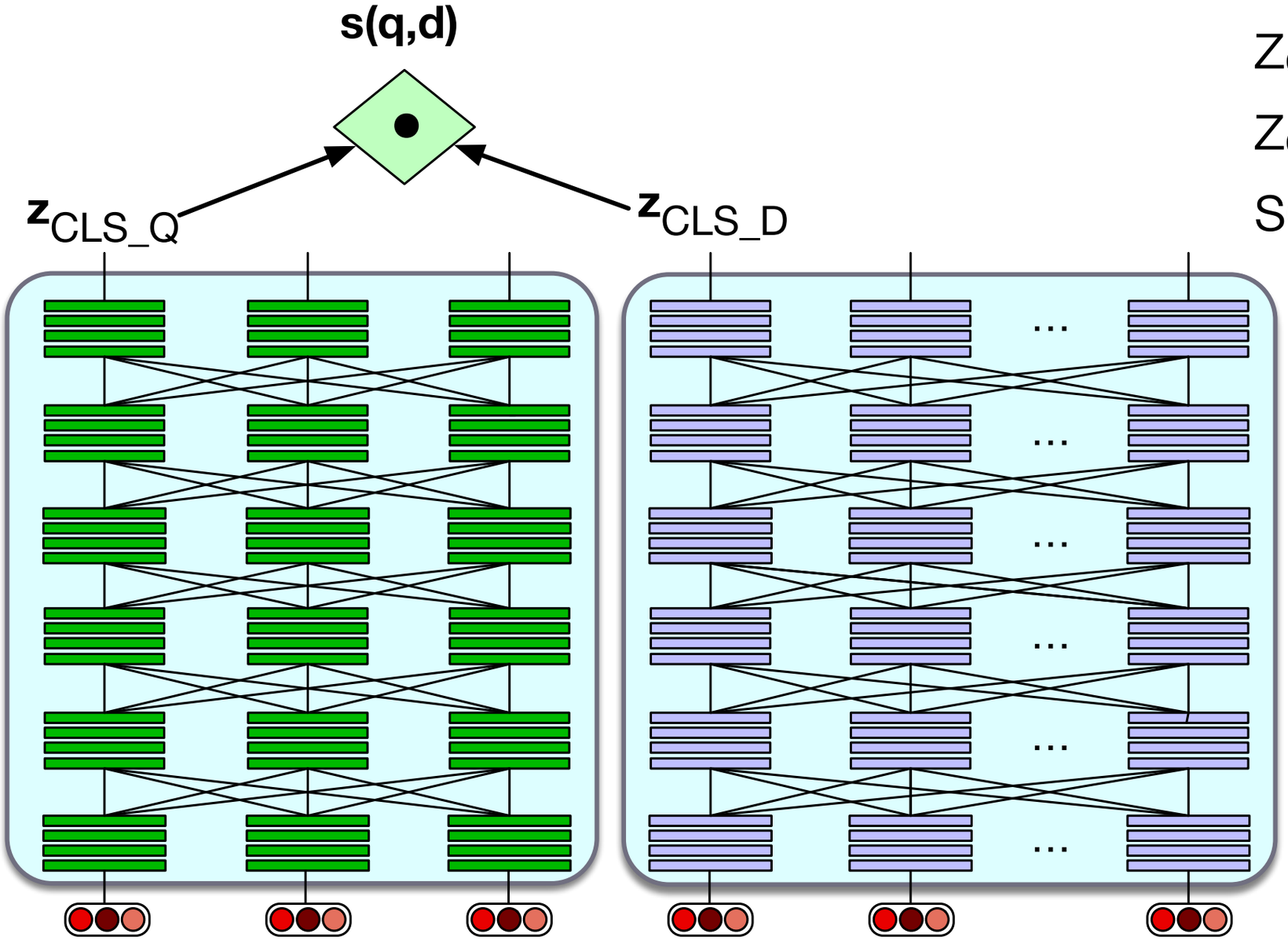
score(q, d) = softmax(U(z))

Dense retrieval #1: Single encoder

- System is run on passages (say 100 tokens) instead of whole documents
- **Training:**
  - BERT and linear layer U can then fine-tuned for relevance
  - Creating a tuning dataset of relevant and non-relevant passages.



Dense retrieval #2: biencoder



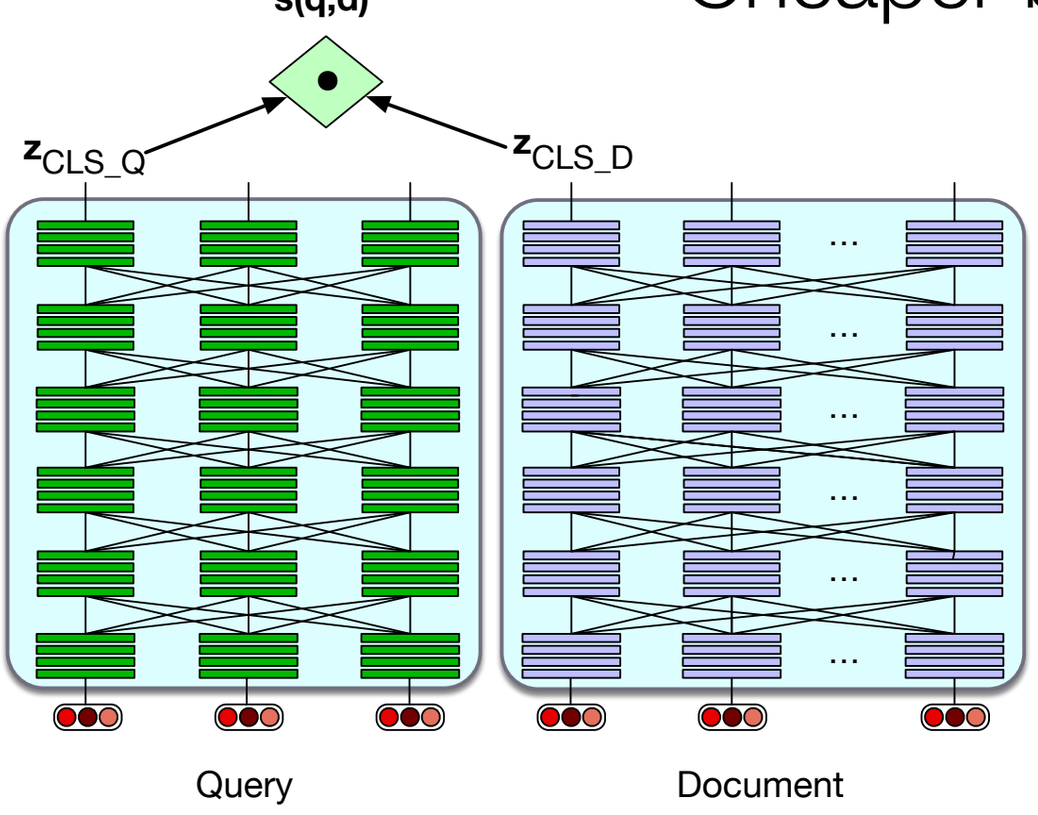
$z_q = BERT_Q(q)[CLS]$

$z_d = BERT_D(d)[CLS]$

$score(q, d) = z_q \cdot z_d$

Dense retrieval #2 (biencoder)

- Encode doc vectors in advance.
- Encode query when it arrives
  - Score is dot product between query vector and precomputed doc vector
  - Cheaper but less accurate



In-between dense retrieval

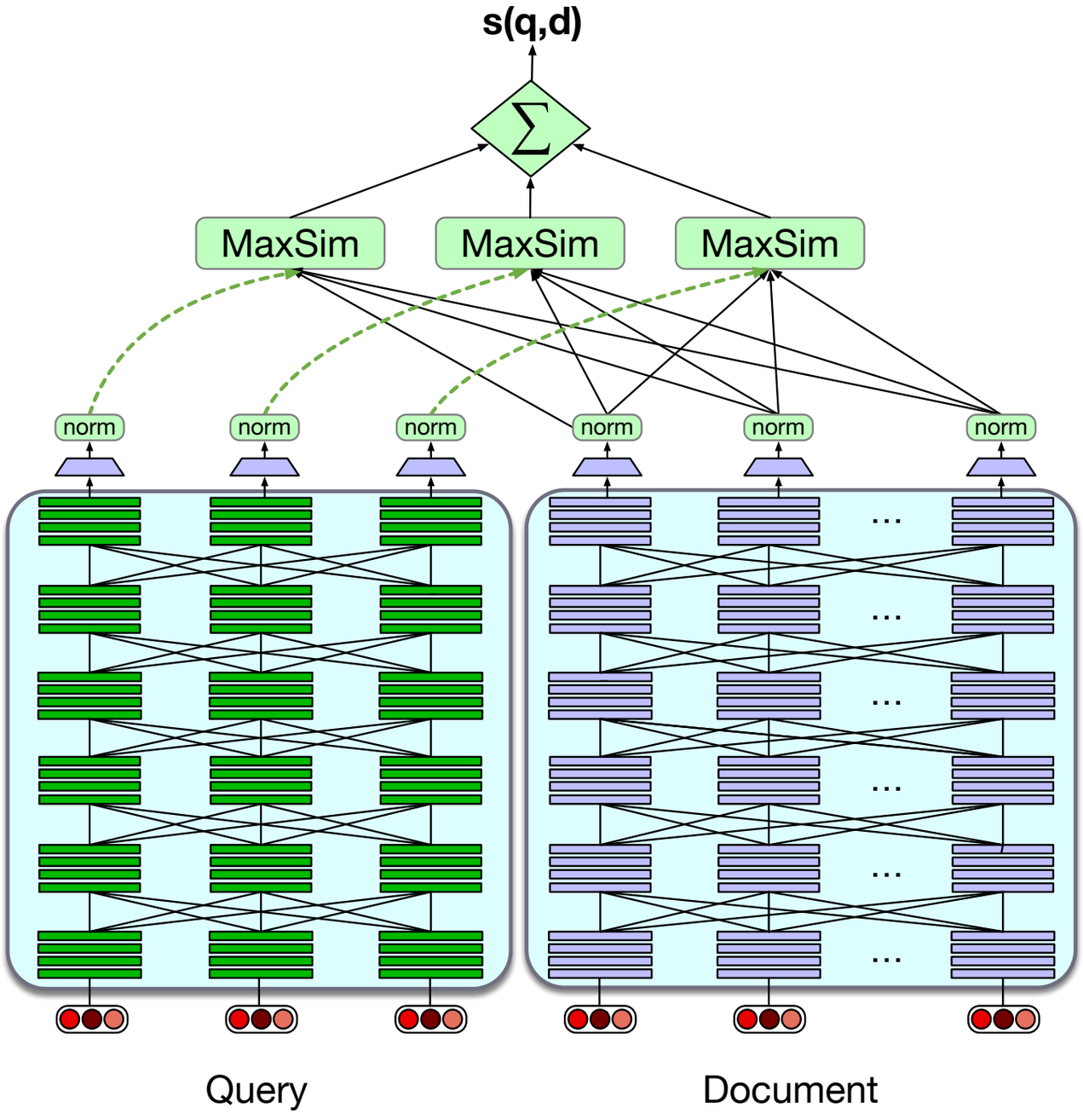
- Use cheap sparse retrieval methods as first pass relevance ranking for each document,
- Then just rerank the top N ranked docs,
  - Using expensive methods like the full BERT scoring

ColBERT

- Precompute document but store each word vector
- Then do maxsim between document and query words

Khattab and Zaharia (2020), Khattab et al (2021)

ColBERT



$$\text{score}(q,d) = \sum_{i=1}^{N} \max_{j=1}^{m} E_{q_i} \cdot E_{d_j}$$

For each query word embedding, compute the maximal similarity (dot product) to any document word embedding. Sum up those similarities

Training for dense retrieval

- ColBERT and other models need to be trained
  - Fine-tune the BERT encoders and train the linear layers (and the special [Q] and [D] embeddings)
  - On datasets of triples $\langle q, d+, d- \rangle$

Efficiency in dense retrieval

- We must rank **every document** for its similarity to the query
- **Efficiency for sparse word-count vectors:** inverted index
- **Efficiency for dense retrieval:**
  - **nearest neighbor search**: finding the set of dense document vectors that have the highest dot product with a dense query vector.
  - Approximate nearest neighbor algorithm **Faiss** (Johnson et al., 2017).
    - Approximates the doc vector by a smaller vector

Back to Retrieval-Augmented Generation

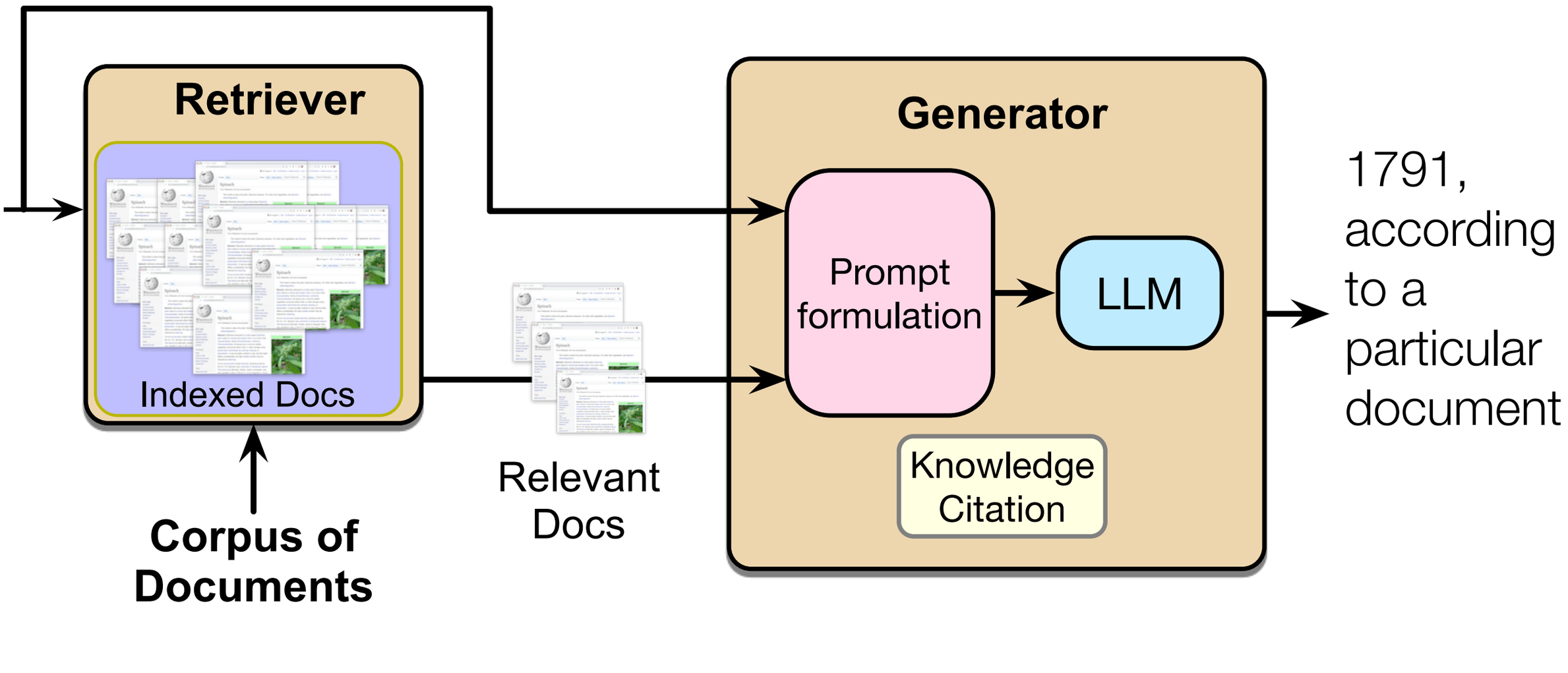
Retrieval-augmented generation

1.Use IR to retrieve documents from some collection

2.Then use LLM to generate an answer conditioned on the documents

Retrieval-augmented generation: overall architecture

User prompt: When was the premiere of The Magic Flute?



Basic RAG

- Given a document collection D and a user query q
  - Call a retriever to return top k passages
  - Create a prompt that includes q and the passages
    “Based on these texts, answer this question:”
  - Call an LLM with the prompt

Basic RAG

$$p(x_1, \ldots, x_n) = \prod_{i=1}^{n} P(x_i \mid R(q); \text{Based on these texts, answer the following question; } x_{<i})$$

Next token to generate depends on:

- R(q) = d1, …, dm, documents retrieved for query q
- prompt “Based on these texts, answer…”
- tokens generated so far

Extensions: Agent-based

Instead of running RAG automatically on every user turn:

- Have a retrieval agents
- System decides when to call it and for which document collection

Extensions: Training

- **Instruction-tune** an LLM on a dataset of questions with retrieved passages and correct answers

- **Test-time compute**: prompt an LLM to answer the question and simultaneously to generate reflections on which passages were useful

Extensions: Knowledge Citations

“Write an answer for the given question using only the provided search results (some of which might be irrelevant) and cite them properly... Always cite for any factual claim".

RAG in Scientific fact-checking

“Write an answer for the given question using only the provided search results (some of which might be irrelevant) and cite them properly... Always cite for any factual claim”.

- Forcing the LLM to always rely on retrieval is extremely useful in scientific fact-checking

- We want the system to answer based on existing research papers, not make stuff up

RAG in Scientific fact-checking

Forcing the LLM to always rely on retrieval is extremely useful in scientific fact-checking

But:

- Turned out to be impossible to get access to journal paper archives. We could only use freely available texts, e.g. arXiv
- Quandary: LLMs forced to always rely on existing papers are unwilling to extrapolate
  - Different experimental conditions than reported previously: Can the result still be expected to hold?

Fact-checking plus retrieval: One example

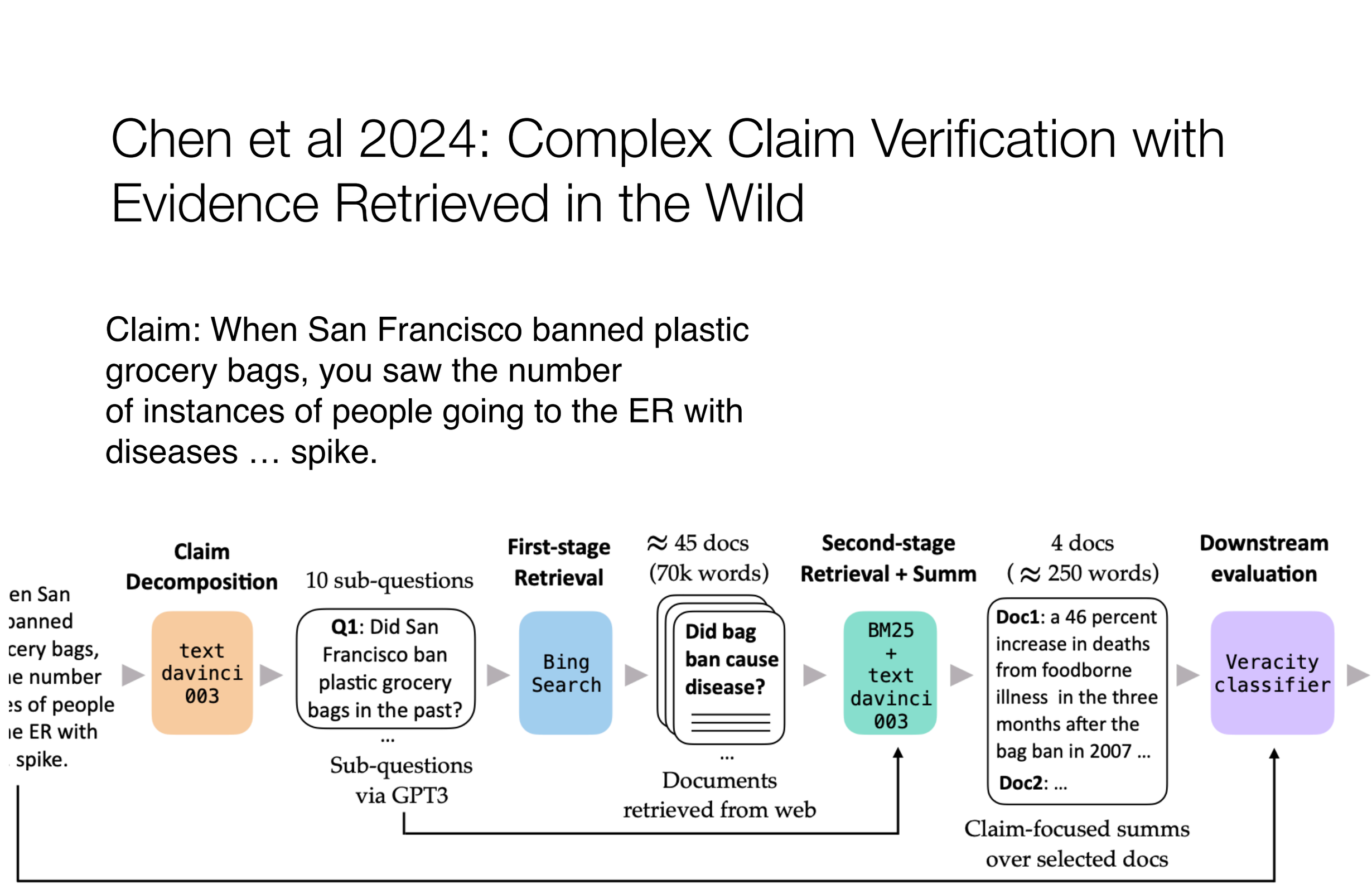
Chen et al 2024: Complex Claim Verification with Evidence Retrieved in the Wild

paper link: https://aclanthology.org/2024.naacl-long.196.pdf

- Checking complex political claims
- Realistic setting: retrieving evidence from the web
  - Use only documents that existed prior to the construction of the claim dataset
  - Don’t use fact-checking websites as documents

Chen et al 2024: Complex Claim Verification with Evidence Retrieved in the Wild

Claim: When San Francisco banned plastic grocery bags, you saw the number of instances of people going to the ER with diseases … spike.



A: Mostly False

Chen et al 2024: Complex Claim Verification with Evidence Retrieved in the Wild

1. Decompose claim into smaller yes-no questions

    - Ask LLM, few-shot prompting with four selected input-decomposition pairs from existing human annotation

    - Multiple rounds of sampling until 10 different questions are obtained

Chen et al 2024: Complex Claim Verification with Evidence Retrieved in the Wild

2. Query commercial search engine with the sub-claims from step 1

- temporal constraint: don’t use search results from after the claim dataset was made
- site constraint: don’t use fact-checking websites
- How reproducible are the results? Their finding:
  - Only 30% of search output URLs overlap when queried two months apart.
  - Final classification result is not strongly impacted

Chen et al 2024: Complex Claim Verification with Evidence Retrieved in the Wild

Step 2 yields a large number of documents. In each document, most of the content is not relevant to the query

3. Second-stage retrieval: Pick most claim-relevant text spans from retrieved documents.

- Segment the document into text spans
- Use sparse retrieval to retrieve top-k highest-scored text spans
  - If two text spans overlap, merge them

Chen et al 2024: Complex Claim Verification with Evidence Retrieved in the Wild

Selected spans are still too long to be fed into an LLM.

4. Prompt an LLM to summarize each selected span separately with respect to the claim

- Few-shot prompting substantially reduces risk of hallucination

5. Fine-tune DeBERTa to do 6-way classification:
true, mostly true, half true, barely true, false, pants-on-fire

- using summaries from step 4